



INGENIERÍA EN AUTOMATIZACIÓN Y ROBOTICA

ASIGNATURA: MICROCONTROLADORES

CODIGO: AAUT103

UNIDAD DE APRENDIZAJE: LCD, TECLADOS MATRICIALES, INTERRUPCIONES Y TIMERS.

CRITERIO DE EVALUACIÓN: Evaluar y seleccionar diferentes funciones de librerías pre-definidas, así como fórmulas de cálculo de tiempos de desborde de timers, para la programación e implementación de aplicaciones que empleen pantallas LCD, teclados matriciales y/o interrupciones varias.

TRABAJO COLABORATIVO N°3

MICROCONTROLADORES

MANEJO DE
ENTRADAS Y SALIDAS

- Mariana José Córdova

(20.669.897-7)

- Alex Díaz

(20.807.586-1)

TRABAJO COLABORATIVO N°3

1. OBJETIVOS:

- Reconocer programación en Lenguaje C, Simulación y Montaje para microcontroladores.
- Manejo de Display y LCD.

2. PASOS PREVIOS:

Revisar y Comprender las Clases para realizar este trabajo colaborativo.

3. INICIO DE LA ACTIVIDAD:

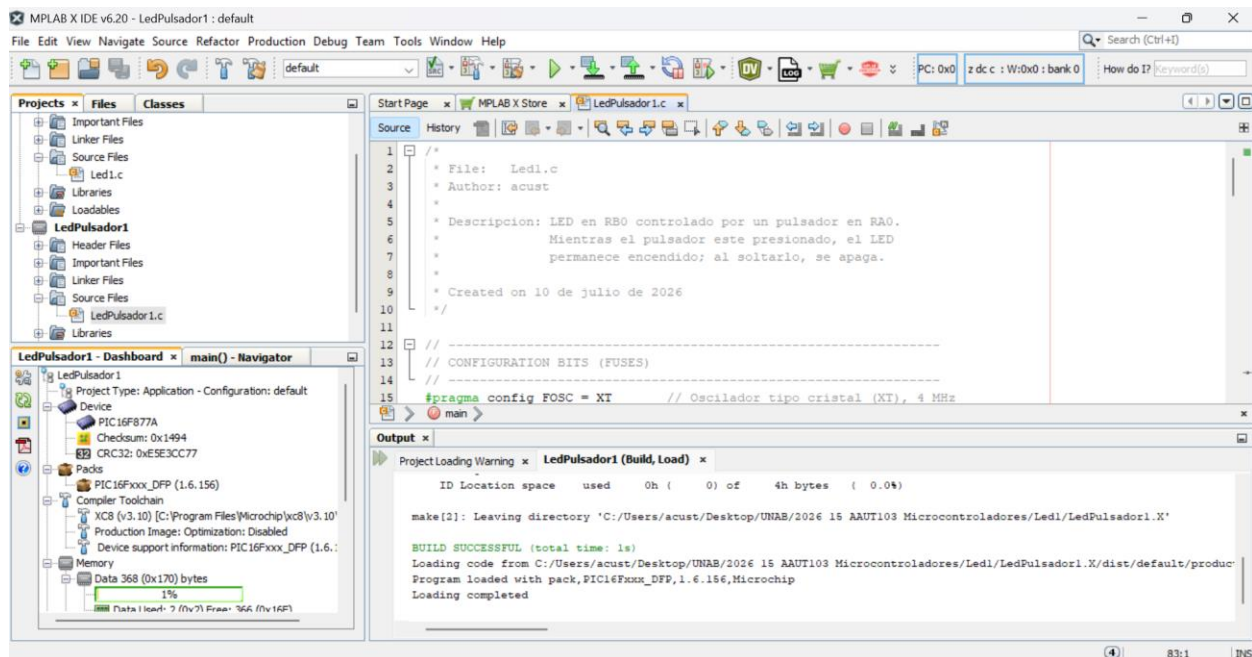
Se realizarán las siguientes actividades.

3.1. INSTRUCCIONES:

En equipo de trabajo establecido se realizarán las siguientes actividades, revisando las exposiciones teóricas y respondiendo las preguntas planteadas. Las actividades (montaje físico) deben estar validadas por el profesor para ser posteriormente evaluadas, luego suba este documento con sus respectivas respuestas al ambiente de aprendizaje.

3.2. PREPARACIÓN PARA LAS ACTIVIDADES DE LABORATORIO:

Revisar la clase sobre el manejo de MPLAB X y el compilador XC8 para PIC



3.3. ACTIVIDAD 1: Control de DISPLAY, cada segmento. (2 puntos)

El visualizador de siete segmentos (llamado también display de 7 segmentos) es una forma de representar caracteres en equipos electrónicos. Está compuesto de siete segmentos que se pueden encender o apagar individualmente. Cada segmento tiene la forma de una pequeña línea. Se podría comparar a escribir números con cerillas o fósforos de madera.

El visualizador de 7 segmentos es un componente que se utiliza para la representación de caracteres (normalmente números) en muchos dispositivos electrónicos, debido en gran medida a su simplicidad. Aunque externamente su forma difiere considerablemente de un led típico, internamente están constituidos por una serie de leds con unas determinadas conexiones internas, estratégicamente ubicados de tal forma que forme un número '8'.

Cada uno de los segmentos que forman la pantalla están marcados con siete primeras letras del alfabeto ('a'-'g'), y se montan de forma que permiten activar cada segmento por separado, consiguiendo formar cualquier dígito numérico.

A continuación, se muestran algunos ejemplos:

Si se activan o encienden todos los segmentos se forma el número "8".

Si se activan sólo los segmentos: "a, b, c, d, e, f," se forma el número "0".

Si se activan sólo los segmentos: "a, b, g, e, d," se forma el número "2".

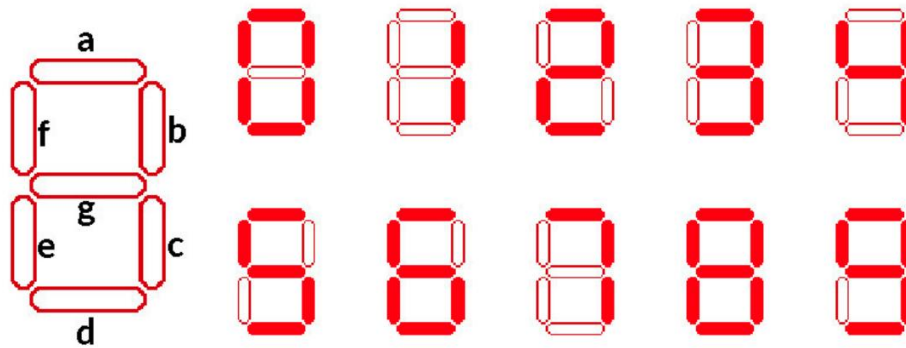
Si se activan sólo los segmentos: "b, c, f, g," se forma el número "4".

Los hay de dos tipos: **ánodo común** y **cátodo común**.

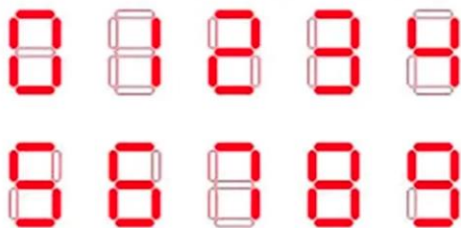
En los de tipo de **ánodo común**, todos los ánodos de los leds o segmentos están unidos internamente a una patilla común que debe ser conectada a potencial positivo (nivel "1"). El encendido de cada segmento individual se realiza aplicando potencial negativo (nivel "0") por la patilla correspondiente a través de una resistencia que limite el paso de la corriente.

En los de tipo de **cátodo común**, todos los cátodos de los leds o segmentos están unidos internamente a una patilla común que debe ser conectada a potencial negativo (nivel "0"). El encendido de cada segmento individual se realiza aplicando potencial positivo (nivel "1") por la patilla correspondiente a través de una resistencia que limite el paso de la corriente.

Caracteres del display 7 segmentos

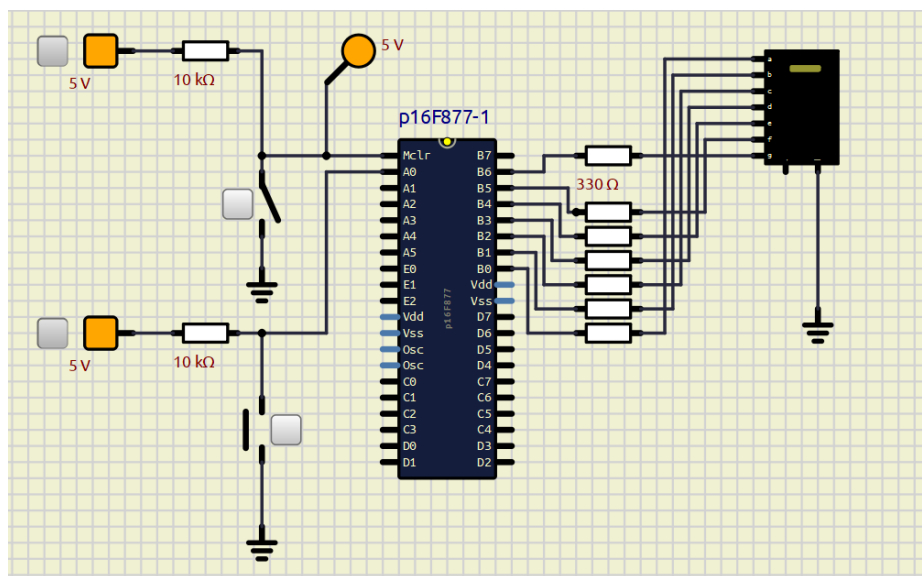
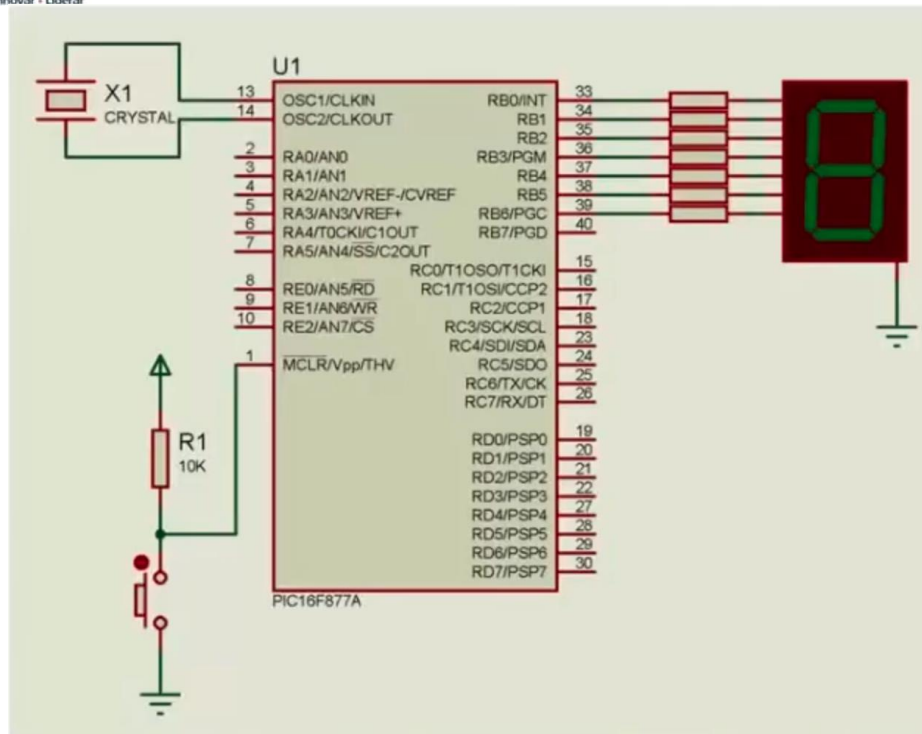


Instrucciones para la programación del microcontrolador Caso Cátodo Común:



0: 00111111 -> 0x3F
 1: 00000110 -> 0x06
 2: 01011011 -> 0x5B
 3: 01001111 -> 0x4F
 4: 01100110 -> 0x66
 5: 01101101 -> 0x6D
 6: 01111101 -> 0x7D
 7: 00000111 -> 0x07
 8: 01111111 -> 0x7F
 9: 01101111 -> 0x6F

En el Proteus o SimulID realizar el siguiente montaje para encender un Display de 7 segmentos Cátodo Común:



En el MPLAB compilar y generar el archivo .hex del siguiente programa:

```

/*
 * File: Led1.c
 * Author: acust
 */

```

- * Descripción: Programa de DIAGNOSTICO. Enciende un solo
- * segmento del display a la vez, recorriendo
- * RB0, RB1, RB2... hasta RB7, con una pausa
- * entre cada uno. Sirve para identificar
- * visualmente que segmento (a,b,c,d,e,f,g,dp)
- * corresponde a cada pin fisico.
- *
- * Created on 10 de julio de 2026
- * /

```
#pragma config FOSC = XT
#pragma config WDTE = OFF
#pragma config PWRTE = ON
#pragma config BOREN = ON
#pragma config LVP = OFF
#pragma config CPD = OFF
#pragma config WRT = OFF
#pragma config CP = OFF

#include <xc.h>
#define _XTAL_FREQ 4000000

void main(void) {

    ADCON1 = 0x06;    // Todos los pines AN como digitales
    TRISB = 0x00;    // PORTB completo como salida
    PORTB = 0x00;    // Apagamos todo al iniciar

    unsigned char i;

    while (1) {

        // Recorremos bit por bit: RB0, RB1, RB2 ... RB7
        for (i = 0; i < 8; i++) {

            PORTB = 0x00;    // Apagamos todos los segmentos
            PORTB = (1 << i);    // Encendemos SOLO el pin RB(i)

            __delay_ms(1000);    // 1 segundo para observar bien
        }
    }
}
```

Mostrar al docente el funcionamiento en Proteus o SimulID. Pegar aquí el resultado del compilador MPLAB y el simulador.

```
*/
* descripcion: programa de diagnostico para un display
* de 7 segmentos de catodo comun
*
* enciende un solo segmento a la vez recorriendo
* rb0, rb1, rb2... hasta rb7
*/

// configuracion del pic
#pragma config FOSC = XT
#pragma config WDTE = OFF
#pragma config PWRTE = ON
#pragma config BOREN = ON
#pragma config LVP = OFF
#pragma config CPD = OFF
#pragma config WRT = OFF
#pragma config CP = OFF

// libreria principal para usar el pic con xc8
#include <xc.h>

// frecuencia del cristal
#define _XTAL_FREQ 4000000

void main(void)
{
    // deja los pines analogicos configurados como digitales
    ADCON1 = 0x06;

    // todo el puerto b sera salida
    TRISB = 0x00;

    // parte con todos los segmentos apagados
    PORTB = 0x00;

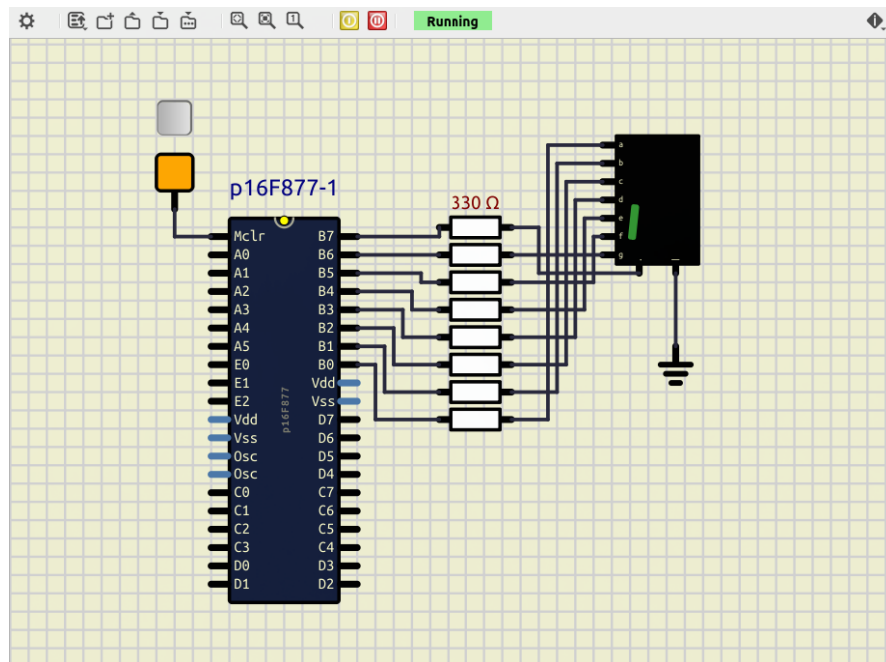
    // variable para recorrer rb0 hasta rb7
    unsigned char i;
```

```
// el programa se repite para siempre
while(1)
{
    // recorre los 8 bits del puerto b
    for(i = 0; i < 8; i++)
    {
        // apaga todos los segmentos
        PORTB = 0x00;

        // enciende solamente el pin rb correspondiente
        PORTB = (1 << i);

        // espera 1 segundo
        __delay_ms(1000);
    }
}

return;
```



3.4. ACTIVIDAD 2: Control de DISPLAY, mostrar números. (2 puntos)

En el MPLAB compilar y generar el archivo .hex de un programa que al pulsar un interruptor, encienda y apague un led. Luego simularlo en Proteus o SimuIiD

Este sería el programa:

```
/*  
* File: Led1.c  
* Author: acost  
*  
* Descripcion: Contador de 0 a 9 en display de 7 segmentos  
*             catodo comun. Cada vez que se presiona el  
*             pulsador conectado a RA0, el display avanza  
*             al siguiente digito (y vuelve a 0 despues del 9).  
*  
* Mapeo de segmentos (PORTB):  
*   RB0 = a  RB1 = b  RB2 = c  RB3 = d  
*   RB4 = e  RB5 = f  RB6 = g  RB7 = dp  
*  
* Created on 10 de julio de 2026  
*/  
  
// -----  
// CONFIGURATION BITS (FUSES)  
// -----
```

```
#pragma config FOSC = XT    // Oscilador tipo cristal (XT), 4 MHz
#pragma config WDTE = OFF    // Watchdog Timer deshabilitado
#pragma config PWRTE = ON    // Power-up Timer habilitado
#pragma config BOREN = ON    // Brown-out Reset habilitado
#pragma config LVP = OFF    // Programacion de bajo voltaje
                             // deshabilitada
#pragma config CPD = OFF    // Proteccion de memoria de datos
                             // deshabilitada
#pragma config WRT = OFF    // Proteccion de escritura en Flash
                             // deshabilitada
#pragma config CP = OFF     // Proteccion de codigo deshabilitada
```

```
// -----
```

```
// LIBRERIAS
```

```
// -----
```

```
#include <xc.h>
```

```
#define _XTAL_FREQ 4000000    // Cristal de 4 MHz
```

```
// -----
```

```
// DEFINICION DE PINES
```

```
// -----
```

```
#define PULSADOR    RA0    // Pulsador conectado a RA0 (PORTA)
```

```
// -----
```

```
// TABLA DE SEGMENTOS (0 al 9) - Catodo comun
```

```
// -----
```

```
// Orden de bits: dp g f e d c b a (coincide con el cableado actual)
```

```
const unsigned char display[10] = {
```

```
    0x3F, // 0
```

```
    0x06, // 1
```

```
    0x5B, // 2
```

```
    0x4F, // 3
```

```
    0x66, // 4
```

```
    0x6D, // 5
```

```
    0x7D, // 6
```

```
    0x07, // 7
```

```
    0x7F, // 8
```

```
    0x6F  // 9
```

```
};
```

```
// -----
```

```
// FUNCION PRINCIPAL
```

```
// -----
```

```
void main(void) {
```

```
    // Variable que guarda el dígito actual mostrado (0 a 9)
```

```
    unsigned char contador = 0;
```

```
    // -----
```

```
    // CONFIGURACION DEL MODULO ADC (OBLIGATORIO por usar  
    RA0)
```

```
    // -----
```

```
    ADCON1 = 0x06;
```

```
    // -----
```

// CONFIGURACION DE DIRECCION DE PINES (TRIS)

// -----

TRISB = 0x00; // PORTB completo como SALIDA (display)

TRISAbits.TRISA0 = 1; // RA0 = ENTRADA (pulsador)

// Mostramos el dígito inicial (0)

PORTB = display[contador];

// -----

// BUCLE PRINCIPAL

// -----

while (1) {

 // Logica negada: pulsador sin presionar -> pull-up

 // externo mantiene RA0 en 1. Al presionar, RA0 = 0.

 if (PULSADOR == 0) {

 // Pequeña espera para el anti-rebote (debounce)

 __delay_ms(20);

 // Confirmamos que sigue presionado despues de la espera

 if (PULSADOR == 0) {

 // Avanzamos al siguiente dígito

 contador++;

 // Si llegamos a 10, volvemos a 0

 if (contador == 10) {

```
        contador = 0;
    }

    // Actualizamos el display con el nuevo digito
    PORTB = display[contador];

    // Esperamos a que el estudiante SUELTE el boton,
    // para no contar varias veces con una sola pulsacion
    while (PULSADOR == 0) {
        // No hacemos nada, solo esperamos
    }

    // Pequeña espera adicional despues de soltar (debounce)
    __delay_ms(20);
}
}
}

return;
}
```

Programa realizado:

```
/*
// descripcion: contador de 0 a 9 en display de 7 segmentos
de catodo comun

conexiones usadas en simulide:
```

```
rb0 -> a
rb1 -> b
rb2 -> c
rb3 -> d
rb4 -> e
rb5 -> f
rb6 -> g
rb7 -> dp
```

cada vez que se presiona el pulsador conectado a ra0
el display avanza al siguiente numero

```
*/
```

```
// configuracion del pic
```

```
#pragma config FOSC = XT
```

```
#pragma config WDTE = OFF
```

```
#pragma config PWRTE = ON
```

```
#pragma config BOREN = ON
```

```
#pragma config LVP = OFF
```

```
#pragma config CPD = OFF
```

```
#pragma config WRT = OFF
```

```
#pragma config CP = OFF
```

```
#include <xc.h>
```

```
#define _XTAL_FREQ 4000000
```

```
// pulsador conectado a ra0 #define PULSADOR RA0
```

```
/*
```

```
tabla para display de 7 segmentos de catodo comun
```

mapeo:

```
rb0 = a
rb1 = b
rb2 = c
rb3 = d
rb4 = e
rb5 = f
rb6 = g
rb7 = dp
```

en catodo comun:

1 = segmento encendido

0 = segmento apagado

*/

```
const unsigned char display[10] = { 0x3F, // 0 0x06, // 1 0x5B, // 2
                                     0x4F, // 3 0x66, // 4 0x6D, // 5 0x7D, // 6 0x07, // 7 0x7F,
                                     // 8 0x6F // 9 };
```

```
void main(void) { // variable que guarda el numero actual unsigned
                  char contador = 0;
```

```
// deja los pines analogicos como digitales
```

```
    ADCON1 = 0x06;
```

```
    // portb completo como salida para el display
```

```
    TRISB = 0x00;
```

```
    // ra0 como entrada para el pulsador
```

```
    TRISAbits.TRISA0 = 1;
```

```
    // muestra inicialmente el numero 0
```

```
PORTB = display[contador];

while(1)
{
    // con pull-up:
    // boton suelto = 1
    // boton presionado = 0
    if(PULSADOR == 0)
    {
        // espera corta para evitar rebotes
        __delay_ms(20);

        // confirma que el boton sigue presionado
        if(PULSADOR == 0)
        {
            // avanza al siguiente numero
            contador++;

            // despues del 9 vuelve al 0
            if(contador == 10)
            {
                contador = 0;
            }

            // actualiza el numero mostrado
            PORTB = display[contador];

            // espera hasta que se suelte el boton
            while(PULSADOR == 0)
            {
```

```

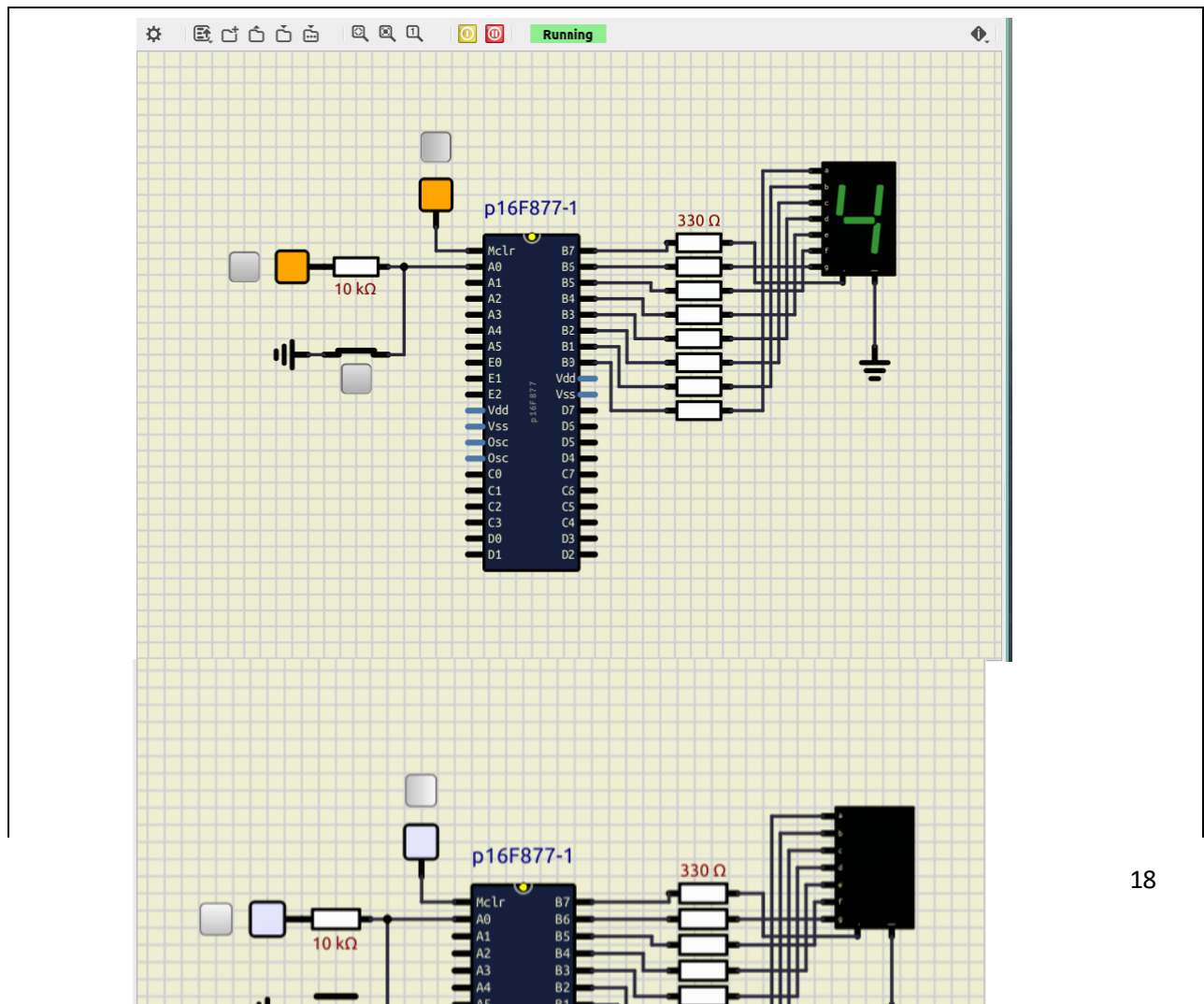
    // solo espera
  }

  // pequeña espera despues de soltar
  __delay_ms(20);
}
}
}

return;
}

```

Mostrar al Docente y colocar aquí un capture de la Simulación: LED Encendido y LED Apagado:



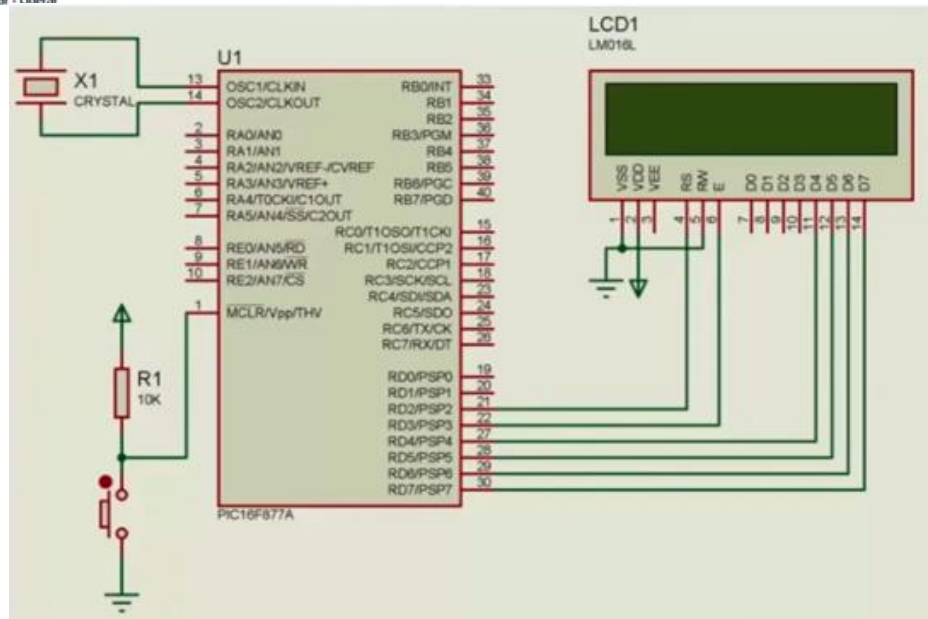
3.5. ACTIVIDAD 3: Manejo LCD. (3 puntos)

Una pantalla de cristal líquido nombrada por sus siglas en inglés Liquid Crystal Display, que se utiliza para ver imágenes fijas y en movimiento. Formada por gran cantidad de píxeles que consisten en moléculas de cristal líquido contenidas entre dos conjuntos de electrodos transparentes. Los cristales líquidos reaccionan de maneras predecibles cuando se cambia la carga eléctrica que circula entre esos electrodos, lo que significa que se tuercen y se mueven de forma que permiten diferentes cantidades (y colores) de luz a través de los cristales.

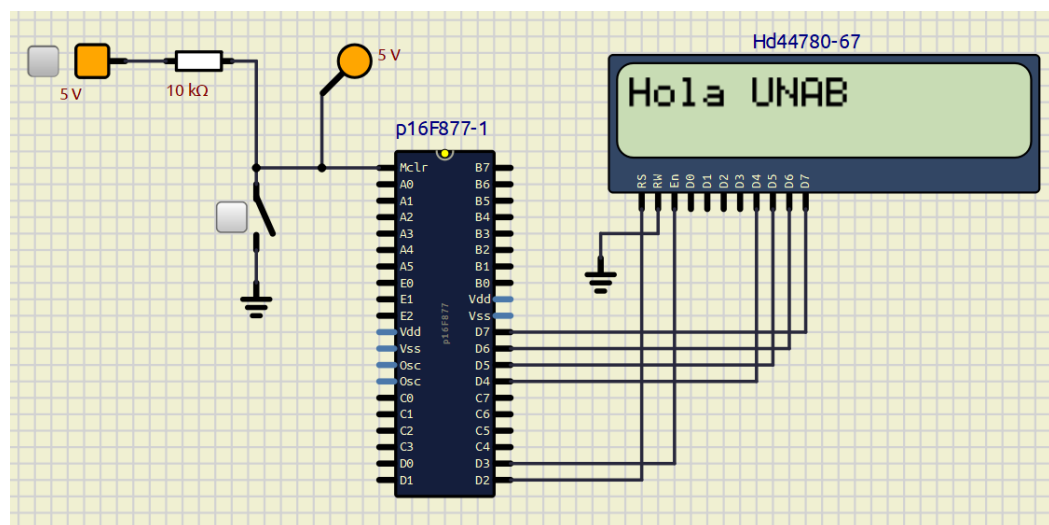
En el Proteus o SimulID realizar el siguiente montaje para el LCD:

Proteus: LM016L → LCD 16x2 (14 pines, sin backlight)

Pin del LCD	Pin del PIC	Función
RS	RD2	Selecciona Comando (0) o Dato (1)
R/W	GND (tierra)	Fijamos siempre en modo "escritura"
E (Enable)	RD3	Pulso que confirma cada envío
D4	RD4	Dato (bit 4)
D5	RD5	Dato (bit 5)
D6	RD6	Dato (bit 6)
D7	RD7	Dato (bit 7)
VSS	GND	Tierra
VDD	+5V	Alimentación
V0	Potenciómetro (contraste)	Divisor de voltaje a GND/VDD



Y en SimulID es el Hd44780



En el MPLAB compilar y generar el archivo .hex de un programa que muestre un mensaje en el LCD. Luego simularlo en Proteus o SimulID.

```
* File:  Led1.c
* Author:  acust
*
* Descripcion: Control de LCD 16x2 (HD44780) en modo 4 bits,
*             todo conectado en PORTD, sin usar librerias
*             externas.
*
* Conexion (todo en PORTD):
*  RS -> RD2    E -> RD3    R/W -> GND (fijo)
*  D4 -> RD4    D5 -> RD5    D6 -> RD6    D7 -> RD7
*
* Created on 10 de julio de 2026
*/
```

```
#pragma config FOSC = XT
#pragma config WDTE = OFF
#pragma config PWRTE = ON
#pragma config BOREN = ON
#pragma config LVP = OFF
#pragma config CPD = OFF
#pragma config WRT = OFF
#pragma config CP = OFF
```

```
#include <xc.h>
#define _XTAL_FREQ 4000000
```

```
// -----
```

```
// DEFINICION DE PINES DE CONTROL DEL LCD (todos en PORTD)
```

```
// -----
```

```
#define LCD_RS    PORTDbits.RD2
```

```
#define LCD_EN    PORTDbits.RD3
```

```
// Los pines de datos (RD4-RD7) se manejan directamente sobre
```

```
// el nibble alto de PORTD dentro de las funciones de envio.
```

```
// -----
```

```
// PULSO DE ENABLE
```

```
// -----
```

```
// El LCD "lee" los datos en el flanco de bajada de la señal E.
```

```
void LCD_Pulso(void) {
```

```
    LCD_EN = 1;
```

```
    __delay_us(50);
```

```
    LCD_EN = 0;
```

```
    __delay_us(50);
```

```
}
```

```
// -----
```

```
// ENVIA UN NIBBLE (4 bits) por D4-D7
```

```
// -----
```

```
void LCD_EnviarNibble(unsigned char nibble) {
```

```
    // Limpiamos SOLO el nibble alto de PORTD (RD4-RD7).
```

```
    // Esto no afecta RD2 (RS) ni RD3 (E), que estan en el
```

```
    // nibble bajo, ni RD0/RD1 que no se usan.
```

```
    PORTD &= 0x0F;
```

```
// Colocamos el nuevo nibble en la parte alta del puerto
PORTD |= (nibble << 4);

LCD_Pulso();
}

// -----
// ENVIA UN COMANDO al LCD (ej: limpiar pantalla, posicionar cursor)
// -----
void LCD_Comando(unsigned char comando) {

    LCD_RS = 0; // RS = 0 -> modo COMANDO

    LCD_EnviarNibble(comando >> 4); // Nibble alto primero
    LCD_EnviarNibble(comando & 0x0F); // Luego nibble bajo

    __delay_ms(2); // Tiempo de procesamiento del LCD
}

// -----
// ENVIA UN DATO (caracter) para mostrar en pantalla
// -----
void LCD_Dato(unsigned char dato) {

    LCD_RS = 1; // RS = 1 -> modo DATO (caracter)

    LCD_EnviarNibble(dato >> 4);
```

```
LCD_EnviarNibble(dato & 0x0F);

__delay_ms(2);
}

// -----
// INICIALIZACION DEL LCD (secuencia obligatoria del fabricante)
// -----

void LCD_Inicializar(void) {

    __delay_ms(20);    // Espera a que el LCD encienda estable

    // Secuencia especial de arranque en modo 4 bits (datasheet HD44780)
    LCD_EnviarNibble(0x03);
    __delay_ms(5);
    LCD_EnviarNibble(0x03);
    __delay_ms(5);
    LCD_EnviarNibble(0x03);
    __delay_ms(5);
    LCD_EnviarNibble(0x02); // Confirma modo 4 bits

    LCD_Comando(0x28); // Modo 4 bits, 2 lineas, fuente 5x8
    LCD_Comando(0x0C); // Display ON, cursor OFF, sin parpadeo
    LCD_Comando(0x06); // Modo de incremento automatico del cursor
    LCD_Comando(0x01); // Limpia pantalla
    __delay_ms(2);
}
```

```
// -----  
  
// ESCRIBE UNA CADENA DE TEXTO COMPLETA  
  
// -----  
  
void LCD_EscribirTexto(const char *texto) {  
    while (*texto != '\0') {  
        LCD_Dato(*texto);  
        texto++;  
    }  
}  
  
  
// -----  
  
// FUNCION PRINCIPAL  
  
// -----  
  
void main(void) {  
  
    ADCON1 = 0x06;  // PORTA y PORTE como digitales (buena practica)  
  
    TRISD = 0x00;  // PORTD completo como salida (LCD)  
  
    LCD_Inicializar();  
  
    LCD_EscribirTexto("Hola UNAB");  
  
    while (1) {  
        // El mensaje queda fijo en pantalla  
    }  
  
    return;
```

Programa realizado:

```
*/ descripcion: muestra un mensaje en un lcd 16x2
* hd44780 usando modo de 4 bits
*
* conexiones:
*
* rs -> rd2
* e  -> rd3
* rw -> gnd
* d4 -> rd4
* d5 -> rd5
* d6 -> rd6
* d7 -> rd7
*/

// configuracion del pic
#pragma config FOSC = XT
#pragma config WDTE = OFF
#pragma config PWRTE = ON
#pragma config BOREN = ON
#pragma config LVP = OFF
#pragma config CPD = OFF
#pragma config WRT = OFF
#pragma config CP = OFF

#include <xc.h>
```

```
#define _XTAL_FREQ 4000000

// pines de control del lcd
#define LCD_RS PORTDbits.RD2
#define LCD_EN PORTDbits.RD3

// genera un pulso en enable
void LCD_Pulso(void)
{
    LCD_EN = 1;
    __delay_us(50);

    LCD_EN = 0;
    __delay_us(50);
}

// envia 4 bits al lcd usando rd4-rd7
void LCD_EnviarNibble(unsigned char nibble)
{
    // limpia solamente rd4-rd7
    PORTD &= 0x0F;

    // coloca el nibble en la parte alta de portd
    PORTD |= (nibble << 4);

    LCD_Pulso();
}
```

```
// envia un comando al lcd
void LCD_Comando(unsigned char comando)
{
    // rs = 0 significa comando
    LCD_RS = 0;

    // envia primero los 4 bits superiores
    LCD_EnviarNibble(comando >> 4);

    // luego envia los 4 bits inferiores
    LCD_EnviarNibble(comando & 0x0F);

    __delay_ms(2);
}

// envia un caracter al lcd
void LCD_Dato(unsigned char dato)
{
    // rs = 1 significa dato
    LCD_RS = 1;

    LCD_EnviarNibble(dato >> 4);
    LCD_EnviarNibble(dato & 0x0F);

    __delay_ms(2);
}
```

```
// inicializa el lcd
void LCD_Inicializar(void)
{
    // espera a que el lcd encienda correctamente
    __delay_ms(20);

    // secuencia de inicio para modo de 4 bits
    LCD_EnviarNibble(0x03);
    __delay_ms(5);

    LCD_EnviarNibble(0x03);
    __delay_ms(5);

    LCD_EnviarNibble(0x03);
    __delay_ms(5);

    LCD_EnviarNibble(0x02);

    // modo 4 bits, 2 lineas, caracteres 5x8
    LCD_Comando(0x28);

    // display encendido, cursor apagado
    LCD_Comando(0x0C);

    // cursor avanza automaticamente
    LCD_Comando(0x06);

    // limpia la pantalla
```

```
LCD_Comando(0x01);
```

```
__delay_ms(2);
```

```
}
```

```
// escribe una cadena de texto completa
```

```
void LCD_EscribirTexto(const char *texto)
```

```
{
```

```
    while(*texto != '\0')
```

```
    {
```

```
        LCD_Dato(*texto);
```

```
        texto++;
```

```
    }
```

```
}
```

```
void main(void)
```

```
{
```

```
    // deja los pines analogicos como digitales
```

```
    ADCON1 = 0x06;
```

```
    // portd completo como salida para el lcd
```

```
    TRISD = 0x00;
```

```
    // inicializa el lcd
```

```
    LCD_Inicializar();
```

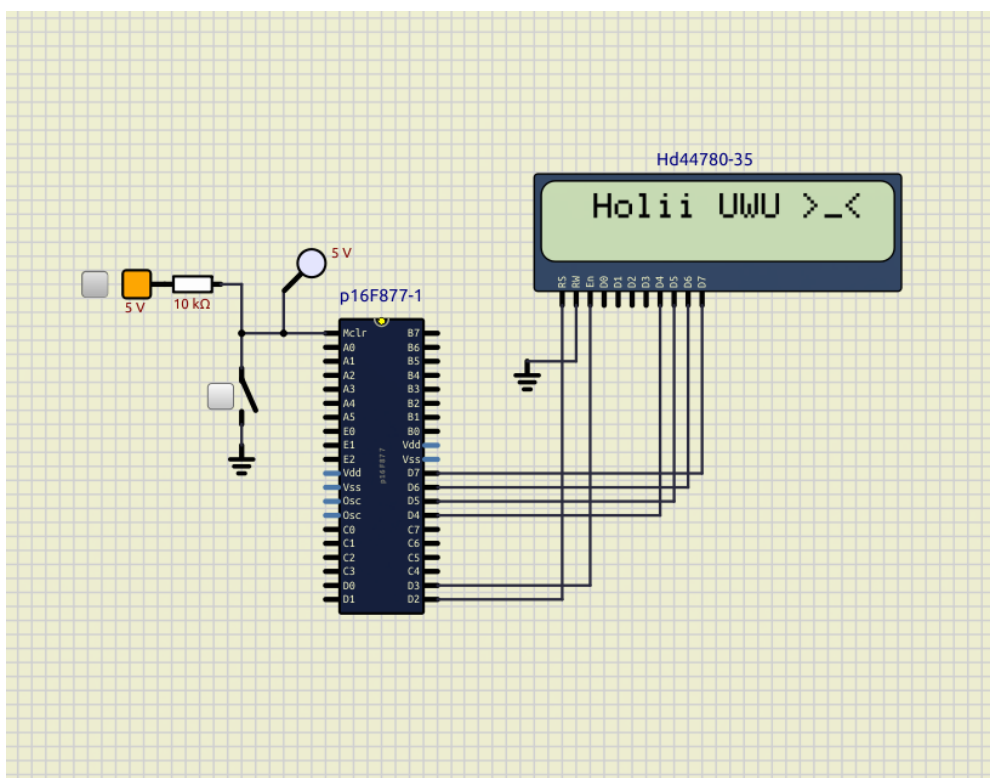
```
    // mensaje que se mostrara en pantalla
```

```
    LCD_EscribirTexto(" Holii UWU >_< ");
```

```
while(1)
{
    // el mensaje queda fijo
}

return;
}
```

Mostrar al Docente y colocar aquí un capture de la Simulación



Concluya sobre el trabajo realizado en este trabajo colaborativo 3

En este trabajo pudimos entender mejor cómo se relaciona el código con las conexiones realizadas en el PIC16F877A. Durante las actividades tuvimos algunos errores, por ejemplo con el orden de los segmentos del display y con el pulsador conectado a RA0, ya que al dejar una entrada sin un estado bien definido el contador no funcionaba como esperábamos. También aprendimos a trabajar con el LCD 16x2 en modo de 4 bits, entendiendo mejor la función de pines como RS, Enable y las líneas de datos. En general, los problemas que tuvimos durante las simulaciones nos ayudaron a revisar cada conexión con más cuidado y a comprender mejor por qué el programa y el montaje deben coincidir para que el circuito funcione correctamente.

Valor ____/7 pts. (1 pto. Cada actividad, para un total de 7 pts.)